

Algorithmique 1 — TP 3

Tableaux · UE Info1.Algo1 · L1 Informatique, UPS · Semestre 1 2026-2027

Les exercices sont présentés en deux temps : **Partie 1 — Énoncés** (le code fourni tel quel dans les fichiers .py, sans les complétions demandées) puis, après un saut de page, **Partie 2 — Corrigé** (les fonctions complétées et chaque fonction exécutée réellement contre les jeux de test fournis).

Rappels — le type abstrait tableau

- Le **nombre d'éléments** d'un tableau est **fixe** (déterminé dès la création).
- Tous les éléments d'un tableau sont de **même type**.
- Chaque élément est repéré par sa **position** dans le tableau, son **indice**.

indices :	0	1	2	3	4	...	n-2	n-1
éléments :	9	0	-13	8	5	...	-20	4

Ainsi les seules opérations autorisées sont les suivantes :

- **Créer** un tableau en précisant sa **longueur** et la **valeur par défaut** (ce qui précise aussi le type des éléments) : `tableau = creer_tableau(longueur, default)`
- **Déterminer la longueur** d'un tableau : `len(tableau)`
- **Lire** un élément à un indice valide : `valeur = tableau[i]`
- **Écrire** un élément à un indice valide : `tableau[i] = valeur`

Les indices i valides sont ceux qui vérifient $0 \leq i < \text{len}(\text{tableau})$.

Toutes les opérations de **modification de la structure** (`append`, `insert`, `del`, `remove`...) ainsi que les **opérations enrichies** (`clear`, `sorted`, `reverse`...) du type natif Python `list` sont donc **interdites** sur ce TP.

Exemple :

```
def creer_tableau(longueur, default):
    assert longueur >= 0, 'Pré-condition'
    return [default]*longueur

# Creation d'un tableau d'entiers de longueur 20 :
tableau = creer_tableau(20, 0)
# Ecriture dans le tableau :
tableau[0] = int(input())
# Lecture et ecriture :
tableau[1] = 2*tableau[0]
# Lecture de la longueur :
n = len(tableau)
# Parcours des elements : i va de 0 a n-1
for i in range(n):
    tableau[i] = tableau[i] + 1
```

Partie 1 — Énoncés

Exercice 1 — création et parcours ★

Fichier `ex01_creation_parcours.py` :

```
def creer_tableau(longueur, default):
    assert longueur >= 0, 'Pré-condition'
    ...

def calculer_somme(tableau):
    ...

def calculer_moyenne(tableau):
    ...
```

- 1) Écrire l'implémentation de `creer_tableau` qui accepte un entier positif ou nul `longueur` ainsi qu'un paramètre scalaire `default` et retourne un tableau de la longueur correspondante dont tous les éléments sont égaux à `default`.
- 2) Compléter `calculer_somme` (somme des valeurs du tableau) et `calculer_moyenne` (moyenne des valeurs, en appelant `calculer_somme`).

Exercice 2 — recherche ★

Même si certaines structures de données sont plus adaptées à la recherche d'un élément, on a parfois besoin de déterminer si une valeur est présente dans un tableau.

Fichier `ex02_recherche.py` :

```
def chercher_premier_indice(tableau, valeur):
    ...

def chercher_dernier_indice(tableau, valeur):
    ...
```

- 1) Compléter `chercher_premier_indice(tableau, valeur)` : retourne le **premier** indice de tableau correspondant à `valeur`, ou -1 si elle est absente.
- 2) Compléter `chercher_dernier_indice(tableau, valeur)` : mêmes paramètres, mais retourne le **dernier** indice correspondant (ou -1 si absente).

Exercice 3 — étendue d'une série statistique ★

L'**étendue** d'une série statistique est la différence entre la valeur la plus grande et la valeur la plus petite de cette série.

Exemple : l'étendue de `[30,20,10,20,30,40]` est égale à `40-10`, c'est-à-dire 30.

Fichier `ex03_etendue.py` :

```
def calculer_etendue(tableau):
    ...
```

- Compléter `calculer_etendue` qui calcule et retourne l'étendue d'une série statistique passée en paramètre.

Contrainte : il est interdit d'utiliser les fonctions `min` et `max` de Python, ou tout autre équivalent.

Exercice 4 — le chat, la souris et la chaîne de caractères ★

Une chaîne de caractères (`str`) peut être lue (mais pas modifiée) comme un tableau de caractères, avec les deux seules opérations autorisées : `len(chaine)` et `chaine[i]`.

Exemple de chaîne : `..C....s...` — le caractère `'C'` représente un chat, `'s'` une souris, le reste est constitué de `'.'`.

Fichier `ex04_chats_souris.py` :

```
def chat_attrape_souris_1(chaine):
    ...

def chat_attrape_souris_2(chaine, saut):
    ...
```

- 1) chaîne contient **exactement un** caractère 'C' et **exactement un** caractère 's'. Le chat saute d'**au plus 4 positions** (gauche ou droite) ; il attrape la souris s'il arrive exactement à sa position. Compléter `chat_attrape_souris_1` qui retourne True si le chat attrape la souris en un saut, False sinon.
- Exemple : C s $\rightarrow$ distance $6 > 4 \rightarrow$ False. C . . s $\rightarrow$ distance $4 \leq 4 \rightarrow$ True.
- 2) Version générique `chat_attrape_souris_2(chaîne, saut)` : chaîne contient **un ou zéro** 'C', **un ou zéro** 's' (False si l'un des deux manque) ; saut précise le nombre de positions franchissables.

Exercice 5 — somme glissante ★★

- Les fonctions de cet exercice doivent respecter le **type abstrait tableau**.
- Elles ne doivent avoir **aucun effet** sur le tableau passé en paramètre.
- Elles doivent retourner un **nouveau tableau** créé exclusivement avec `creer_tableau`.

Fichier `ex05_somme_glissante.py` :

```
def creer_tableau(longueur, default):
    assert longueur >= 0, 'Pré-condition'
    return [default] * longueur

def somme_deux_a_deux(tableau):
    assert len(tableau) >= 2, 'Pré-condition'
    ...

def somme_glissante(tableau, n):
    assert len(tableau) >= n, 'Pré-condition'
    ...
```

- 1) Compléter `somme_deux_a_deux` : prend un tableau d'entiers d'au moins deux éléments et retourne un nouveau tableau des sommes deux à deux. Exemple : `[1,2,3,4]  $\rightarrow$  [3,5,7]`, c'est-à-dire `[1+2, 2+3, 3+4]`.
- 2) Généraliser en `somme_glissante(tableau, n)` : retourne le tableau des sommes de `n` éléments consécutifs. Exemple sur `[1,2,3,4]` : si `n=3  $\rightarrow$  [6,9]` ; si `n=4  $\rightarrow$  [10]`.
- 3) Une implémentation naïve de la question 2 peut *bloquer* (sans erreur) sur le test de la version optimisée. Objectif : ne pas recalculer chaque somme intégralement — calculer la première somme, puis chaque somme suivante à partir de la précédente en retranchant/ajoutant les 2 nombres appropriés.

Exemple (fenêtre de taille 4 sur `[1,3,6,9,5,4,2]`) :

fenêtre [0;4[	1	3	6	9	5	4	2
- 1							
+ 5							
fenêtre [1;5[	1	3	6	9	5	4	2

Plutôt que de calculer séparément `1+3+6+9` et `3+6+9+5`, on retranche 1 puis on ajoute 5.

Application : en divisant chaque somme par `n`, on obtient une **moyenne glissante**, utile pour lisser des données à forte variation locale (ex. température heure par heure — voir le Corrigé).

Exercice 6 — fusion de tableaux triés ★★

Algorithme fondamental du *tri fusion* (étudié plus tard) : fusionner deux tableaux déjà triés pour obtenir un nouveau tableau trié.

- Les fonctions doivent respecter le **type abstrait tableau** et n'avoir **aucun effet** sur les tableaux passés en paramètres.
- Tout nouveau tableau doit être créé exclusivement avec `creer_tableau`.

Fichier `ex06_fusionner_tableaux_tries.py` :

```
def creer_tableau(longueur, default):
    assert longueur >= 0, 'Pré-condition'
    return [default]*longueur

def est_croissant(tableau):
    ...

def fusionner_tableaux_tries(tableau1, tableau2):
    assert est_croissant(tableau1) and est_croissant(tableau2), 'Pré-condition'
    ...
```

- 1) Compléter `est_croissant(tableau)` : True si `tableau` est trié par ordre croissant au sens large, False sinon.
- 2) Compléter `fusionner_tableaux_tries(tableau1, tableau2)` (deux tableaux triés en entrée) : retourne un tableau trié, fusion des deux. Exemple : `[1,3,3]` et `[-6,2,4]` → `[-6,1,2,3,3,4]`.

Algorithme : dans une boucle principale, on parcourt simultanément `tableau1` et `tableau2` (indices `i1/i2`) ; à chaque étape on choisit le plus petit des deux éléments lus, on l'écrit à l'indice `i1+i2` du résultat, on incrémente l'indice correspondant. Une fois un tableau entièrement parcouru, on complète avec le tableau restant.

Déroulé sur `tableau1=[1,3,3]`, `tableau2=[-6,2,4]` :

<code>i1=0 i2=0</code>	1	3	3	-6	2	4	-6					
<code>i1=0 i2=1</code>	1	3	3	-6	2	4	-6	1				
<code>i1=1 i2=1</code>	1	3	3	-6	2	4	-6	1	2			
<code>i1=1 i2=2</code>	1	3	3	-6	2	4	-6	1	2	3		
<code>i1=2 i2=2</code>	1	3	3	-6	2	4	-6	1	2	3	3	
<code>i1=3 i2=2</code>	1	3	3	-6	2	4	-6	1	2	3	3	4

Exercice 7 — permutation circulaire ★★

Une **permutation circulaire** (ou cycle) est une liste d'indices tous distincts. Si le cycle `[2,5,1]` est appliqué au tableau `[11,12,13,14,15,16]` : l'élément d'indice 2 (13) est envoyé à l'indice 5, l'élément d'indice 5 (16) à l'indice 1, l'élément d'indice 1 (12) à l'indice 2. Résultat : `[11,16,12,14,15,13]`.

On peut effectuer les modifications en partant de la fin du cycle (l'élément sauvegardé ou déplacé laisse sa place pour celui qu'il remplace) :

initial	11	12	13	14	15	16
12 sauvegardé	11		13	14	15	16
16 → indice 1	11	16	13	14	15	
13 → indice 5	11	16		14	15	13
12 → indice 2	11	16	12	14	15	13

Remarque : un cycle vide, ou de longueur 1, ne modifie pas le tableau.

Fichier `ex07_appliquer_cycle.py` :

```
# Fonction de permutation (Question 2)
def permuter(tableau, i, j):
    n = len(tableau)
    assert 0 <= i < n and 0 <= j < n, 'Indices invalides'
    tableau[i], tableau[j] = tableau[j], tableau[i]

def appliquer_cycle(tableau, cycle):
    ...
```

- 1) Compléter `appliquer_cycle(tableau, cycle)` qui modifie `tableau` selon le principe ci-dessus (respecter le type abstrait `tableau`).
- 2) On fournit `permuter(tableau, i, j)` qui échange les éléments d'indices `i` et `j` (garantit qu'aucun élément n'est perdu ni dupliqué). Modifier `appliquer_cycle` pour que ses modifications soient **exclusivement** réalisées via `permuter`.

Exercice 8 — chapeau blanc ★

Objectifs : découvrir qu'une interface peut cacher différentes implémentations ; tester les fonctionnalités de `list` autorisées ou non par le type `tableau` défini en cours ; s'amuser à mettre en défaut l'implémentation, en mode « *white hat* ».

Fichier `ex08_creer_tableau.py` (donné, à ne pas modifier) :

```
class creer_tableau():
    def __init__(self, longueur, default):
        assert longueur >= 0, 'Pré-condition'
        self._longueur = longueur
        self._elements = [default]*longueur
    def __len__(self):
        return self._longueur
    def __getitem__(self, i):
        assert 0 <= i < self._longueur
        return self._elements[i]
    def __setitem__(self, i, valeur):
        assert 0 <= i < self._longueur and type(valeur) == type(self._elements[0])
        self._elements[i] = valeur
    def __repr__(self):
        return repr(self._elements)
    def __iter__(self):
        assert False
    def __dir__(self):
        assert False
    __dict__ = {}
```

Fichier `ex08_chapeau_blanc.py` :

```
from ex08_creer_tableau import *

# PROGRAMME PRINCIPAL
tableau = creer_tableau(10, 0)
# print(tableau)
for i in range(len(tableau)):
    tableau[i] = i
# print(tableau)
for i in range(len(tableau)//2):
    j = len(tableau)-1-i
    tableau[i], tableau[j] = tableau[j], tableau[i]
# print(tableau)
```

- 1) a) Lire le programme principal et prédire ce qu'afficheraient les `print` s'ils étaient décommentés. b) Vérifier en les décommentant. (Indication : les deux fichiers doivent être dans le même répertoire.)
- 2) Tester d'autres fonctionnalités du type `list` natif sur `tableau` : `append`, `pop`, `insert`, `min`, `max`, l'itération sur les éléments...
- 3) ★★★★★ En supposant que l'on ne connaît pas a priori les noms des membres `_longueur`/`_elements` (noms arbitrairement longs, pas de force brute possible) : tenter de contourner l'interface de la classe pour réaliser des opérations permises par `_elements` (de type `list` natif). Si vous y parvenez, indiquez le contournement trouvé à la personne qui encadre le TP (esprit *white hat*).

Partie 2 — Corrigé

Exercice 1 — création et parcours ★

```
def creer_tableau(longueur, default):
    assert longueur >= 0, 'Pré-condition'
    return [default] * longueur

def calculer_somme(tableau):
    somme = 0
    for i in range(len(tableau)):
        somme = somme + tableau[i]
    return somme

def calculer_moyenne(tableau):
    return calculer_somme(tableau) / len(tableau)
```

Vérifié en exécutant `test_creer_tableau`, `test_calculer_somme` et `test_calculer_moyenne` : tout passe.

Exercice 2 — recherche ★

```
def chercher_premier_indice(tableau, valeur):
    for i in range(len(tableau)):
        if tableau[i] == valeur:
            return i
    return -1

def chercher_dernier_indice(tableau, valeur):
    for i in range(len(tableau) - 1, -1, -1):
        if tableau[i] == valeur:
            return i
    return -1
```

On parcourt du début pour le premier indice, de la fin (`range(len(tableau) - 1, -1, -1)`) pour le dernier — même structure, sens inversé. Vérifié sur les 13 assertions de chaque test (dont le tableau vide) : tout passe.

Exercice 3 — étendue d'une série statistique ★

```
def calculer_etendue(tableau):
    if len(tableau) == 0:
        return 0
    mini = tableau[0]
    maxi = tableau[0]
    for i in range(1, len(tableau)):
        if tableau[i] < mini:
            mini = tableau[i]
        if tableau[i] > maxi:
            maxi = tableau[i]
    return maxi - mini
```

Cas particulier à ne pas oublier : tableau vide → le test exige 0, alors que `tableau[0]` lèverait une erreur — d'où le traitement séparé en tout début de fonction. Balayage min/max en une seule passe, sans utiliser `min/max`. Vérifié sur les 9 assertions fournies : tout passe.

Exercice 4 — le chat, la souris et la chaîne de caractères ★

```
def chat_attrape_souris_1(chaine):
    indice_c = -1
    indice_s = -1
    for i in range(len(chaine)):
        if chaine[i]=='C':
            indice_c = i
        if chaine[i]=='s':
            indice_s = i
    distance = indice_c-indice_s
    if distance<0:
        distance = -distance
    return distance<=4
```

On balaie une seule fois la chaîne en retenant le dernier indice vu pour 'C' et pour 's' (chacun apparaît une seule fois dans la version 1), puis on compare la distance absolue à 4.

```
def chat_attrape_souris_2(chaine, saut):
    indice_c = -1
    indice_s = -1
    for i in range(len(chaine)):
        if chaine[i]=='C':
            indice_c = i
        if chaine[i]=='s':
            indice_s = i
    if indice_c==-1 or indice_s==-1:
        return False
    distance = indice_c-indice_s
    if distance<0:
        distance = -distance
    return distance<=saut
```

Même balayage ; les indices restent à -1 si 'C' ou 's' est absent, ce qui déclenche directement le False attendu avant même de calculer une distance. Vérifié sur les 8+17 assertions fournies (les deux versions) : tout passe.

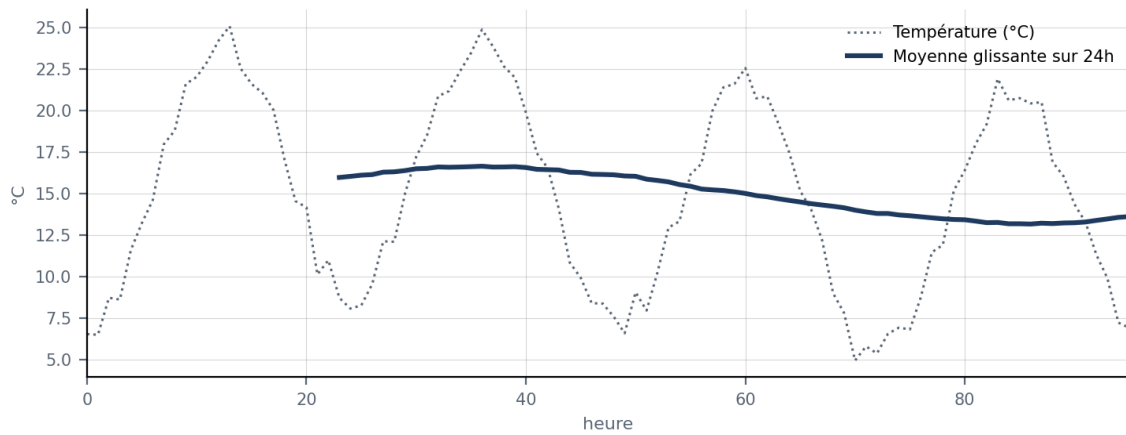
Exercice 5 — somme glissante ★★

```
def somme_deux_a_deux(tableau):
    assert len(tableau)>=2, 'Pré-condition'
    resultat = creer_tableau(len(tableau)-1,0)
    for i in range(len(resultat)):
        resultat[i] = tableau[i]+tableau[i+1]
    return resultat
```

Version optimisée de somme_glissante (question 3) — on calcule la première somme terme à terme, puis chaque somme suivante en retranchant l'élément qui sort de la fenêtre et en ajoutant celui qui y entre :

```
def somme_glissante(tableau,n):
    assert len(tableau)>=n, 'Pré-condition'
    resultat = creer_tableau(len(tableau)-n+1,0)
    somme = 0
    for i in range(n):
        somme = somme+tableau[i]
    resultat[0] = somme
    for i in range(1,len(resultat)):
        somme = somme-tableau[i-1]+tableau[i+n-1]
        resultat[i] = somme
    return resultat
```

Une version naïve (qui recalcule chaque somme intégralement, en $O(n \cdot \text{longueur})$ au total) **pass**e les tests de la question 2 mais **bloque** sur le test de performance de la question 3 : 199999 éléments, fenêtre $n=100000 \rightarrow$ environ 10 milliards d'opérations. La version optimisée ci-dessus ne fait, elle, que deux opérations par somme (une soustraction, une addition) : **vérifiée par exécution**, elle traite ce cas en 0,015 s.



Application — température (données synthétiques, alternance jour/nuit + bruit) sur 96h, et sa moyenne glissante sur 24h calculée avec la fonction ci-dessus : la courbe lissée fait ressortir la tendance multi-jours en gommant l'oscillation jour/nuit.

Exercice 6 — fusion de tableaux triés ★★

```
def est_croissant(tableau):
    for i in range(1, len(tableau)):
        if tableau[i] < tableau[i-1]:
            return False
    return True

def fusionner_tableaux_tries(tableau1, tableau2):
    assert est_croissant(tableau1) and est_croissant(tableau2), 'Pré-condition'
    n1 = len(tableau1)
    n2 = len(tableau2)
    resultat = creer_tableau(n1+n2, 0)
    i1, i2 = 0, 0
    while i1 < n1 and i2 < n2:
        if tableau1[i1] <= tableau2[i2]:
            resultat[i1+i2] = tableau1[i1]
            i1 = i1+1
        else:
            resultat[i1+i2] = tableau2[i2]
            i2 = i2+1
    while i1 < n1:
        resultat[i1+i2] = tableau1[i1]
        i1 = i1+1
    while i2 < n2:
        resultat[i1+i2] = tableau2[i2]
        i2 = i2+1
    return resultat
```

En cas d'égalité ($\text{tableau1}[i1] \leq \text{tableau2}[i2]$), on prend l'élément de `tableau1` en priorité — c'est ce choix qui produit exactement le déroulé de la Partie 1 (le déroulé y trace le même algorithme, sans qu'aucune égalité n'y apparaisse). Vérifié sur les 11+12 assertions fournies (`est_croissant`, `fusionner_tableaux_tries`) : tout passe.

Exercice 7 — permutation circulaire ★★

Question 1 — indexation directe (on sauvegarde l'élément à l'indice de fin de cycle, puis on décale en partant de la fin) :

```
def appliquer_cycle(tableau, cycle):
    if len(cycle) <= 1:
        return None
    sauvegarde = tableau[cycle[-1]]
    for i in range(len(cycle)-1, 0, -1):
        tableau[cycle[i]] = tableau[cycle[i-1]]
    tableau[cycle[0]] = sauvegarde
    return None
```

Question 2 — même décalage, mais réalisé exclusivement avec `permuter` : échanger successivement (`cycle[i]`, `cycle[i-1]`) en partant de la fin produit exactement le même résultat, sans variable de sauvegarde explicite (et le cas `len(cycle) ≤ 1` ne nécessite plus de traitement séparé, la boucle étant alors naturellement vide) :

```
def appliquer_cycle(tableau, cycle):
    for i in range(len(cycle)-1, 0, -1):
        permuter(tableau, cycle[i], cycle[i-1])
    return None
```

Vérifié sur les 4 assertions fournies (cycle vide, longueur 1, et les deux cycles de longueur 3 et 6 du sujet) pour les **deux** versions : tout passe, et les deux produisent bien le même tableau final.

Exercice 8 — chapeau blanc ★

1) Prédiction puis exécution réelle des 3 prints (mêmes valeurs) :

```
1) [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
2) [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
3) [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

La boucle 2 remplit le tableau avec ses indices ; la boucle 3 échange `tableau[i] ↔ tableau[len-1-i]` pour `i` de 0 à `len//2-1`, ce qui renverse le tableau.

2) Fonctionnalités du type `list` testées sur `tableau` — résultat réel obtenu à l'exécution :

```
min(tableau) -> AssertionError (min() itère via __iter__, qui vaut 'assert False')
max(tableau) -> AssertionError (idem)
tableau.append(99) -> AttributeError ('creer_tableau' n'a pas de méthode append)
tableau.pop() -> AttributeError (idem)
for x in tableau: ... -> AssertionError (__iter__ interdit explicitement)
dir(tableau) -> AssertionError (__dir__ interdit explicitement)
vars(tableau) -> {} (le leurre __dict__ = {} masque le vrai contenu)
```

Toutes les opérations non prévues par l'interface échouent bien comme voulu : la classe tient son rôle de type abstrait fermé.

3) ★★★★★ — contournement « white hat » trouvé (exécuté, sans jamais utiliser les noms

`_longueur/_elements`) : le module standard `gc` expose `gc.get_referents(objet)`, qui retourne — au niveau de l'objet Python lui-même, en contournant tout getter/assert défini par la classe — la liste des objets directement référencés par `objet`. Pour un tableau de 10 éléments :

`gc.get_referents(tableau)` retourne `[10, [9, 8, 7, ..., 0], <class ...>]` — la vraie liste interne est là, identifiable sans connaître son nom de champ (il suffit de garder l'élément qui est une `list`). On peut alors l'appendre ou la sortir librement : la longueur déclarée (`len(tableau)`) ne suit pas cette modification, ce qui casse l'invariant du type — la preuve que l'encapsulation Python (préfixe `_`) est une convention, pas une protection. Conformément à l'énoncé, ce contournement est à signaler à l'enseignant plutôt qu'exploité.

```
>>> referents = gc.get_referents(tableau)
>>> referents
[10, [9, 8, 7, 6, 5, 4, 3, 2, 1, 0], <class 'creer_tableau'>]
>>> liste_interne = [r for r in referents if isinstance(r, list)][0]
>>> liste_interne.append(777)
>>> tableau # la valeur 777 est bien là...
[9, 8, 7, 6, 5, 4, 3, 2, 1, 0, 777]
>>> len(tableau) # ...mais la longueur declaree n'a pas bougé
10
```

À retenir

- **Type abstrait avant implémentation** : `creer_tableau/len/lecture/écriture` suffisent à tout écrire sur un tableau — s'y limiter garantit que le code marche aussi avec la classe de l'exercice 8, alors que `list.append` ou `list.sort` ne le permettraient pas.
- **Optimiser une fenêtre glissante** : ne jamais resommer toute la fenêtre — mettre à jour la somme précédente en retranchant l'élément sortant et en ajoutant l'entrant (exercice 5) fait passer un algorithme de bloquant à quasi instantané.
- **Fusionner deux tableaux triés en un seul passage** : deux indices, on avance toujours celui du plus petit élément lu — brique de base du tri fusion (exercice 6).
- **Permutation circulaire** : appliquer un cycle en partant de sa fin, avec des échanges successifs via une seule fonction permuter, garantit qu'aucun élément n'est perdu ni dupliqué (exercice 7).
- **Une interface n'est pas une protection** : des assertions sur `__iter__`/`__dir__` et un leurre `__dict__ = {}` bloquent tous les accès de haut niveau, mais `gc.get_referents()` atteint directement les objets réellement référencés — l'encapsulation par préfixe `_` reste une convention (exercice 8).
- **Toujours traiter le cas vide séparément** quand une valeur par défaut (0 pour l'étendue) ne peut pas s'obtenir en lisant `tableau[0]` (exercice 3).